Universidad Galileo	Guatemala 26 de Octubre del 2026
Facultad: FISICC	**Alumno: Roberto Molina**
Curso: Microprocesadores	**Carnet: 24000849**
Sección: A	**Hora de Laboratorio: 1:00 a 1:50**
Auxiliar: Brandon Mendoza	**Día de Laboratorio: Lunes**

Laboratorio #9 Pulse Width Modulation

Resumen:

En esta práctica se trabajó con los timers del microcontrolador STM32G431 de la tarjeta NUCLEO-G431KB para generar una señal de modulación por ancho de pulso (PWM), y se combinaron con el convertidor analógico-digital (ADC) configurado con interrupciones.

Se usaron dos entradas analógicas: un potenciómetro, para elegir el nivel de brillo, y un fotoresistor, para saber si había luz o no en el ambiente. El ADC se configuró para que avise con una interrupción cada vez que termina una conversión, así el programa principal no tiene que quedarse esperando el resultado y las lecturas se guardan dentro de la rutina de interrupción.

Para la señal PWM se le dio reloj al timer, se configuró el pin del LED en función alternativa para conectarlo al canal del timer y se programaron los registros que definen el período (ARR) y el ancho de pulso (CCR), junto con el modo PWM y la habilitación de la salida. En el ciclo infinito, el valor del potenciómetro se usa para calcular el ancho de pulso, y con eso el brillo del LED. Los otros cuatro LEDs muestran ese mismo nivel, encendiéndose uno por uno conforme se gira la perilla.

El LED controlado por PWM solo se enciende cuando el fotoresistor detecta oscuridad. Si hay luz, el LED queda apagado sin importar la posición del potenciómetro.

Objetivos:

- Configurar un timer del STM32G431 para generar una señal PWM y controlar el brillo de un LED cambiando su ciclo de trabajo.
- Configurar el ADC con interrupciones para leer un potenciómetro y un fotoresistor sin detener el programa principal.
- Combinar ambas lecturas para que el LED de PWM se encienda únicamente en la oscuridad con el brillo elegido, y mostrar ese nivel en cuatro LEDs.

Teoría:

Timers del STM32G4

Los timers son bloques del microcontrolador que trabajan de forma independiente del CPU. Cada uno tiene un contador que avanza con una señal de reloj y puede usarse para medir tiempo, generar interrupciones periódicas, capturar eventos o generar señales PWM [1]. La familia STM32G4 tiene timers de control avanzado (TIM1, TIM8 y TIM20), de propósito general (TIM2 a TIM5 y TIM15 a TIM17) y básicos (TIM6 y TIM7) [1]. No todos los modelos los tienen todos. En el caso del STM32G431, no incluye TIM5 ni TIM20 [1].

Los timers de propósito general y los avanzados tienen varios canales de captura/comparación, y cada canal puede sacar su propia señal PWM por un pin. Los timers básicos no tienen canales de salida, por eso no sirven para generar PWM [1].

Modulación por ancho de pulso (PWM)

Una señal PWM es una onda cuadrada que solo tiene dos niveles, alto y bajo, pero en la que se puede cambiar cuánto tiempo pasa en alto dentro de cada período. El porcentaje del período en el que la señal está en alto se llama ciclo de trabajo (duty cycle) [2]. Con un ciclo de trabajo de 0 % la señal siempre está en bajo, con 50 % se parece a una onda cuadrada ideal y con 100 % siempre está en alto [2].

$$D = \frac{t_{\text{alto}}}{T} \times 100\ \%$$

Si la frecuencia es baja, el ojo alcanza a ver que el LED se enciende y se apaga. Pero si la frecuencia es suficientemente alta, por ejemplo de 100 Hz o más, el LED ya no parpadea a la vista, sino que se ve más tenue que encendido por completo [2]. Así, con un pin digital que solo puede estar en 0 o en 1, se puede controlar el brillo de un LED cambiando únicamente el ciclo de trabajo.

Generación de PWM con un timer

El contador del timer (CNT) avanza con una frecuencia que depende del prescaler (PSC): el reloj del timer se divide entre PSC + 1 [1]. Cuando el contador llega al valor del registro de auto-recarga (ARR), regresa a cero y empieza un nuevo período. Por lo tanto, la frecuencia de la señal PWM es [1]:

$$f_{\text{PWM}} = \frac{f_{\text{TIM}}}{(PSC + 1)(ARR + 1)}$$

En el modo PWM 1, con el contador en modo ascendente, la salida del canal se mantiene en alto mientras CNT es menor que el registro de comparación (CCRx) y pasa a bajo cuando CNT lo alcanza [1]. Es decir, el flanco de subida ocurre al reiniciarse el contador y el flanco de bajada cuando CNT llega a CCRx. Con esto, el ciclo de trabajo queda determinado por CCRx y ARR:

$$D = \frac{CCR_{\mathrm{x}}}{ARR + 1} \times 100\,\%$$

Por ejemplo, con un reloj de 16 MHz, PSC = 15 y ARR = 999, el contador avanza a 1 MHz y la señal tiene una frecuencia de 1 kHz. Con CCRx = 250, el ciclo de trabajo es de 25 %. Si CCRx vale 0, la salida se queda siempre en bajo y el LED permanece apagado.

Registros del timer

Para configurar un canal en modo PWM se utilizan los siguientes registros [1]:

- **RCC_APB1ENR1 / RCC_APB2ENR:** habilitan el reloj de los timers. TIM2, TIM3 y TIM4 están en el bus APB1 (bits 0, 1 y 2 de RCC_APB1ENR1), mientras que TIM1, TIM8, TIM15, TIM16 y TIM17 están en el bus APB2 (bits 11, 13, 16, 17 y 18 de RCC_APB2ENR).
- **TIMx_PSC:** prescaler que divide el reloj del timer.
- **TIMx_ARR:** valor de auto-recarga. Define el período de la señal.
- **TIMx_CCRx:** valor de comparación de cada canal. Define el ancho de pulso.
- **TIMx_CCMRx:** selecciona el modo del canal. En los bits OCxM se escribe 0110 para el modo PWM 1, y el bit OCxPE activa el preload del registro CCRx para que el nuevo valor se cargue al inicio del siguiente período.
- **TIMx_CCER:** el bit CCxE habilita la salida del canal en el pin, y el bit CCxP elige su polaridad.
- **TIMx_CR1:** el bit CEN (bit 0) enciende el contador, DIR (bit 4) elige si cuenta hacia arriba o hacia abajo, CMS (bits 5 y 6) elige el alineamiento de la señal y ARPE (bit 7) activa el preload de ARR.
- **TIMx_EGR:** el bit UG (bit 0) genera un evento de actualización por software, lo que carga los valores de los registros con preload antes de arrancar el contador.

- **TIMx_BDTR:** el bit MOE (bit 15) habilita todas las salidas. Este registro solo existe en los timers avanzados y en TIM15, TIM16 y TIM17, y si no se activa MOE la señal no sale por el pin.

Función alternativa de los pines

Para que la señal del timer salga por un pin, el pin no puede estar en modo de salida normal. En GPIOx_MODER se escribe 10, que corresponde a función alternativa [1]. Luego, en los registros GPIOx_AFRL (pines 0 a 7) y GPIOx_AFRH (pines 8 a 15), cada pin usa cuatro bits para elegir cuál de las funciones alternativas AF0 a AF15 se conecta a él [1]. El número de AF que corresponde a cada canal de cada timer depende del pin, y se obtiene de la tabla de funciones alternativas de la hoja de datos del microcontrolador.

ADC con interrupciones

En el laboratorio anterior el programa se quedaba revisando la bandera EOC hasta que terminaba cada conversión. Con interrupciones, en cambio, el ADC avisa al CPU cuando el resultado está listo, y el CPU puede hacer otras tareas mientras tanto [1]. Para esto se usan los siguientes registros y bits del ADC [1]:

- **ADC_CFGR:** el bit CONT (bit 13) activa el modo continuo, en el que el ADC empieza una nueva conversión en cuanto termina la anterior. Los bits RES eligen la resolución, que con 00 es de 12 bits (0 a 4095).
- **ADC_SQR1:** el campo L indica cuántas conversiones tiene la secuencia, menos uno, y los campos SQ1 (bits 6 a 10) y SQ2 (bits 12 a 16) eligen el canal de la primera y la segunda conversión. Así se pueden leer el potenciómetro y el fotoresistor en una misma secuencia.
- **ADC_IER:** el bit EOCIE (bit 2) genera una interrupción al terminar cada conversión, y el bit EOSIE (bit 3) al terminar toda la secuencia.
- **ADC_ISR:** las banderas EOC (bit 2) y EOS (bit 3) indican cuál de los dos eventos ocurrió. La bandera EOC se limpia sola al leer ADC_DR.

Además de habilitar la interrupción en el ADC, hay que habilitarla en el NVIC, que es el controlador de interrupciones del Cortex-M4. Con CMSIS esto se hace con la función NVIC_EnableIRQ, pasándole el número de la interrupción [3]. En el STM32G431, el ADC1 y el ADC2 comparten la interrupción ADC1_2 [1], y su rutina de servicio debe llamarse exactamente ADC1_2_IRQHandler, que es el nombre que aparece en la tabla de vectores

del archivo de arranque. Si el nombre no coincide, el compilador no da error, pero la interrupción nunca llega a la función.

Las variables que se modifican dentro de la rutina de interrupción y se leen en el ciclo principal se declaran como volatile. Así el compilador siempre vuelve a leer su valor de la memoria, en lugar de usar una copia que pudo haber guardado en un registro del CPU.

Fotoresistor

Un fotoresistor (LDR) es una resistencia que cambia de valor según la luz que recibe. En la oscuridad su resistencia es muy alta, de cientos de kΩ o más, y conforme aumenta la luz va bajando, hasta unos pocos cientos de ohmios con mucha iluminación [4]. Para leerlo con el ADC se conecta en serie con una resistencia fija, formando un divisor de voltaje entre la alimentación y tierra, y el punto medio se lleva a la entrada analógica [4]:

$$V_{\text{out}} = V_{\text{CC}} \frac{R}{R + R_{\text{LDR}}}$$

Con esta conexión, cuando hay luz la resistencia del LDR baja y el voltaje de salida sube, y en la oscuridad el voltaje baja [4]. Si se invierte la posición del LDR y de la resistencia fija, el comportamiento también se invierte. La relación no es lineal con la cantidad de luz, por lo que para saber si está oscuro basta con comparar la lectura del ADC contra un umbral.

Potenciómetro

Como en el laboratorio anterior, el potenciómetro se conecta con sus extremos a 3.3 V y a GND, de modo que el voltaje del pin central varía entre 0 V y 3.3 V conforme se gira la perilla [5]. Con 12 bits, la lectura va de 0 a 4095, y ese valor se puede escalar al rango del registro CCRx para elegir el ciclo de trabajo:

$$CCR_{\text{x}} = N_{\text{ADC}} \times \frac{ARR + 1}{4096}$$

Resultados prácticos:

```c
#include "RTE_Components.h"
#include CMSIS_device_header
#define UMBRAL_OSCURO 1300 // Definir un umbral para determinar si hay suficiente luz ambiente

volatile uint16_t lecturas[2]; // Array para almacenar las lecturas del ADC [0] = potenciometro, [1] = fotoresistor
volatile uint16_t indice = 0; // que canal de la secuencia acaba de llegar


void gpio_init(void);
void adc_init(void);
void tim3_init(void);
void mostrar_nivel(uint16_t nivel);
void ADC1_2_IRQHandler(void);
```

Fig. 1. Definición del umbral, variables y prototipos

```c
void gpio_init(void){
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN; // Habilitar reloj GPIOA
    RCC -> AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // Habilitar reloj GPIOB

    GPIOB -> MODER &= ~(3<<0); // Limpiar bits de modo para PB0
    GPIOB -> MODER |= (2<<0); // Configurar PB0 como funcion alternativa (AF)
    GPIOB -> AFR[0] &= ~(0xF<<0); // Limpiar bits de AF para PB0
    GPIOB -> AFR[0] |= (2<<0); // Configurar PB0 como AF2 (TIM3_CH3)

    GPIOB -> MODER &= ~(3<<8); // Limpiar bits de modo para PB4
    GPIOB -> MODER |= (1<<8); // Configurar PB4 como salida

    GPIOB -> MODER &= ~(3<<10); // Limpiar bits de modo para PB5
    GPIOB -> MODER |= (1<<10); // Configurar PB5 como salida

    GPIOA -> MODER &= ~(3<<24); // Limpiar bits de modo para PA12
    GPIOA -> MODER |= (1<<24); // Configurar PA12 como salida

    GPIOA -> MODER &= ~(3<<22); // Limpiar bits de modo para PA11
    GPIOA -> MODER |= (1<<22); // Configurar PA11 como salida

    GPIOA -> MODER &= ~(3<<0); // Limpiar bits de modo para PA0
    GPIOA -> MODER |= (3<<0); // Configurar PA0 como entrada analógica

    GPIOA -> MODER &= ~(3<<2); // Limpiar bits de modo para PA1
    GPIOA -> MODER |= (3<<2); // Configurar PA1 como entrada analógica
}
```

Fig. 2. Configuración de los pines

```c
void tim3_init(void){
    RCC -> APB1ENR1 |= RCC_APB1ENR1_TIM3EN; // Habilitar reloj TIM3
    TIM3 -> PSC = 3; // Prescaler 16Mhz / 4 = 4Mhz
    TIM3 -> ARR = 4095; // 4 MHz / 4096 = 976.5625 Hz esta es la frecuencia de la señal PWM y su periodo es de 1.024 ms
    TIM3 -> CCR3 = 2048; // 50% duty cycle

    TIM3 -> CCMR2 &= ~((3<<0)|(7<<4)|(1<<16)); // Limpiar CC3S (bits 1:0) y OC3M (bits 6:4 y bit 16) para CH3
    TIM3 -> CCMR2 |= (6<<4); // Configurar CH3 en modo PWM1 OC3M=0110 PWM modo 1
    TIM3 -> CCMR2 |= (1<<3); // Habilitar preload para CH3

    TIM3 -> CCER &= ~(1<<9); // CC3P=0, salida activa en alto
    TIM3 -> CCER |= (1<<8); // Habilitar salida para CH3

    TIM3 -> CR1 &= ~((1<<4)|(3<<5)); // DIR=0, contador ascendente, CMS = 00, edge-aligned mode
    //TIM3 -> CR1 |= (1<<7); // ARPE: precarga habilitada de ARR

    TIM3 -> EGR |= (1<<0); // UG: cargar el valor de ARR y PSC y CCR3 para que el timer arranque con los valores correctos desde el primer ciclo
    TIM3 -> CR1 |= (1<<0); // CEN: arrancar el contador

}
```

Fig. 3. Configuración del TIM3 en modo PWM

```c
void adc_init(void){
    RCC -> AHB2ENR |= RCC_AHB2ENR_ADC12EN; // Habilitar reloj ADC1/ADC2
    ADC12_COMMON -> CCR &= ~(3<<16); // Limpiar bits de CKMODE para seleccionar el reloj del ADC
    ADC12_COMMON -> CCR |= (1<<16); // CKMODE=01, HCLK/1 = 16 MHz

    ADC1 -> CR &= ~(1<<29); // DEEPPWD=0 salir del modo de bajo consumo
    ADC1 -> CR |= (1<<28); // ADVREGEN=1 encender regulador de voltaje del ADC
    for(volatile int i=0; i<10000; i++); // Esperar >= 20us para que el regulador se estabilice

    ADC1 -> CR &= ~(1<<30); // ADCALDIF=0, calibrar en modo single-ended mide el voltaje de un pin respecto a GND
    ADC1 -> CR |= (1U<<31); // ADCAL=1, iniciar calibración
    while (ADC1 -> CR & (1U<<31)); // Esperar a que la calibración termine (ADCAL vuelve a 0)

    ADC1 -> SMPR1 |= (7<<3) | (7<<6); // Tiempo de muestreo maximo (640.5 ADC cycles) para canales 1 y 2

    ADC1 -> SQR1 &= ~((0xF<<0)|(0x1F<<6)|(0x1F<<12)); // Limpiar bits de L, SQ1 y SQ2
    ADC1 -> SQR1 |= (1<<0); //L=1, 2 conversiones en la secuencia (L+1)
    ADC1 -> SQR1 |= (1<<6); // SQ1=1, primer canal en la secuencia es el canal 1 (PA0)
    ADC1 -> SQR1 |= (2<<12); // SQ2=2, segundo canal en la secuencia es el canal 2 (PA1)

    ADC1 -> CFGR &= ~(3<<3); // RES=00, resolución de 12 bits
    ADC1 -> CFGR |= (1<<13); // CONT=1, modo de conversión continua
    ADC1 -> CFGR |= (1<<12); // OVRMOD=1, sobrescritura de datos, el valor de DR se sobrescribe con cada nueva conversión

    ADC1 -> IER |= (1<<2); // EOCIE=1, habilitar interrupción al terminar la conversión de cada canal
    ADC1 -> IER |= (1<<3); // EOSIE=1, habilitar interrupción al terminar la secuencia de conversiones

    NVIC_EnableIRQ(ADC1_2_IRQn); // Habilitar interrupción del ADC en el NVIC

    ADC1 -> ISR = (1<<0); // Limpiar bandera ADRDY escribiendo 1
    ADC1 -> CR |= (1<<0); // ADEN=1 encender el ADC
    while(!(ADC1 -> ISR & (1<<0))); // Esperar a que ADRDY=1, el ADC está listo

    ADC1 -> CR |= (1<<2); // ADSTART=1, iniciar la conversión
}
```

Fig. 4. Configuración del ADC con interrupciones

```c
void ADC1_2_IRQHandler(void){
    uint32_t estado = ADC1 -> ISR; // Leer el registro de estado del ADC

    if(estado & (1<<2)){ //EOC termino una conversion de un canal
        uint16_t dato = ADC1 -> DR; // Leer el dato convertido tambien limpia la bandera EOC
        if(indice < 2){
            lecturas[indice] = dato; // Guardar el dato en el arreglo de lecturas
            indice++; // Incrementar el índice para la siguiente lectura
        }
    }

    if(estado & (1<<3)){ //EOS termino la secuencia de conversiones
        ADC1 -> ISR = (1<<3); // Limpiar la bandera EOS escribiendo 1
        indice = 0; // Reiniciar el índice para la siguiente secuencia
    }
}
```

Fig. 5. Rutina de interrupción del ADC

```c
void mostrar_nivel(uint16_t nivel){
    uint16_t leds_encender = (nivel * 5) / 4096; // 5 niveles parejos, 819 cuentas cada uno
    uint32_t encender_a = 0; // pines de gpioa que se deben encender
    uint32_t encender_b = 0; // pines de gpiob que se deben encender

    if(leds_encender >= 1){
        encender_a |= (1<<12); // Encender LED (PA12)
    }
    if(leds_encender >= 2){
        encender_a |= (1<<11); // Encender LED (PA11)
    }
    if(leds_encender >= 3){
        encender_b |= (1<<5); // Encender LED (PB5)
    }
    if(leds_encender >= 4){
        encender_b |= (1<<4); // Encender LED (PB4)
    }

    GPIOA -> BSRR = (((1<<12)|(1<<11)) << 16) | encender_a; // Bits 16-31 apagan PA12/PA11, bit 0-15 encienden los que toquen
    GPIOB -> BSRR = (((1<<4)|(1<<5)) << 16) | encender_b; // Bits 16-31 apagan PB4/PB5, bit 0-15 encienden los que toquen
}
```

Fig. 6. Indicador de nivel con cuatro LEDs

```c
int main() {
    gpio_init();
    adc_init();
    tim3_init();
    for (;;) {
        if(lecturas[1] < UMBRAL_OSCURO){ // Si la lectura del fotoresistor es menor que el umbral, significa que hay poca luz
            TIM3 -> CCR3 = lecturas[0]; // Ajustar el duty cycle del PWM según la lectura del potenciometro
        }else{
            TIM3 -> CCR3 = 0; // Apagar el LED si hay suficiente luz
        }
        mostrar_nivel(lecturas[0]); // Mostrar el nivel de intensidad con los LEDs según la lectura del potenciometro
    }
}
```

Fig. 7. Programa principal

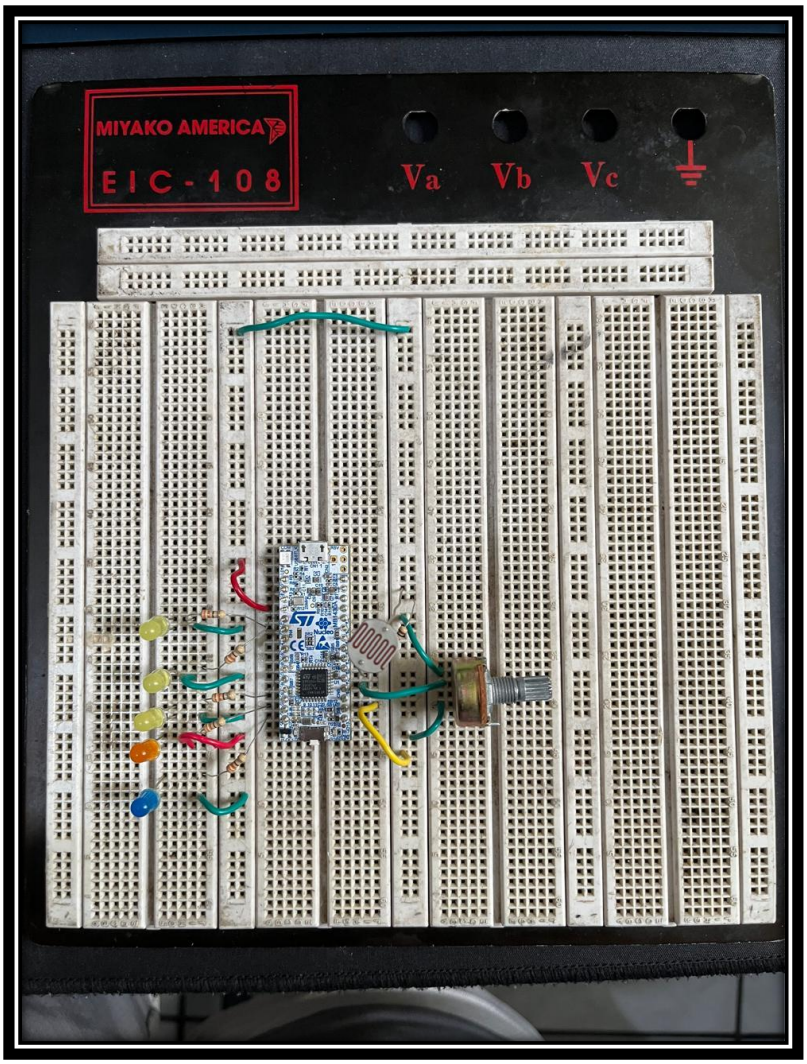


Fig. 8. Circuito

Video demostrativo: https://youtube.com/shorts/6DFLa7xIJl0?si=FWH6Y2tMhggeVAFR

Procedimientos teóricos:

Definiciones y variables globales (Fig. 1)

Al inicio se definió la constante UMBRAL_OSCURO con un valor de 1300, que es el límite para decidir si hay poca luz. También se declaró el arreglo lecturas[2], donde lecturas[0] guarda el valor del potenciómetro y lecturas[1] el del fotoresistor, y la variable indice, que indica cuál de los dos canales acaba de llegar. Ambas se declararon como volatile porque se modifican dentro de la rutina de interrupción y se leen en el ciclo principal. Si no fueran volatile, el compilador podría guardar una copia en un registro del CPU y el programa principal nunca vería los valores nuevos.

Con una referencia de 3.3 V y 12 bits, el umbral equivale aproximadamente a:

$$V = 1300 \times \frac{3.3\ V}{4096} \approx 1.05\ V$$

Configuración de los pines (Fig. 2)

Primero se habilitó el reloj de los puertos A y B en RCC_AHB2ENR. Después se configuró cada pin limpiando sus dos bits de MODER y escribiendo el modo que necesita:

- **PB0 (10, función alternativa):** es la salida del PWM. En AFR[0] (que corresponde a GPIOB_AFRL) se escribió AF2 en los bits 0 a 3, que conecta PB0 con el canal 3 del TIM3 (TIM3_CH3).
- **PA12, PA11, PB5 y PB4 (01, salida):** son los cuatro LEDs que muestran el nivel del potenciómetro.
- **PA0 y PA1 (11, analógico):** son las entradas del ADC. PA0 es el canal 1, donde está el potenciómetro, y PA1 es el canal 2, donde está el fotoresistor.

Limpiar los dos bits antes de escribir es importante en PB4, porque después del reset este pin no arranca en modo analógico como los demás, sino en función alternativa (10), ya que se usa para la interfaz de depuración. Si solo se hiciera el OR con 01, el pin quedaría en 11 (analógico) en lugar de salida.

Configuración del TIM3 (Fig. 3)

Se usó el TIM3, que es un timer de propósito general conectado al bus APB1, por lo que su reloj se habilitó con el bit TIM3EN de RCC_APB1ENR1. El microcontrolador trabaja con el oscilador interno de 16 MHz, y con PSC = 3 el contador avanza a:

$$f_{\text{CNT}} = \frac{f_{\text{CLK}}}{PSC + 1} = \frac{16\ MHz}{3 + 1} = 4\ MHz$$

Para elegir el valor de ARR se despejó la fórmula de la frecuencia del PWM vista en clase:

$$ARR = \frac{f_{\text{CLK}}}{(PSC + 1) f_{\text{PWM}}} - 1$$

Se escogió ARR = 4095 para que el registro CCR3 tenga el mismo rango que la lectura del ADC de 12 bits (0 a 4095). Así, el valor del potenciómetro se puede copiar directamente a CCR3 sin hacer ninguna conversión. Con estos valores la frecuencia de la señal PWM queda en:

$$f_{\text{PWM}} = \frac{16\ 000\ 000}{(3 + 1)(4095 + 1)} = 976.5625\ Hz$$

$$T = \frac{1}{f_{\text{PWM}}} = 1.024\ ms$$

Esta frecuencia es mucho mayor a la que el ojo alcanza a distinguir, por lo que el LED no parpadea a la vista, sino que se ve más o menos brillante según el ciclo de trabajo. Como valor inicial se cargó CCR3 = 2048, que corresponde a la mitad del período:

$$D = \frac{CCR3}{ARR + 1} = \frac{2048}{4096} = 50\ \%$$

El canal 3 se configura en el registro CCMR2, porque CCMR1 corresponde a los canales 1 y 2. Primero se limpiaron los bits CC3S para que el canal sea salida y OC3M, incluyendo el bit 16, que es el cuarto bit del modo en el STM32G4. Luego se escribió 0110 en OC3M para el modo PWM 1 y se activó OC3PE, para que el nuevo valor de CCR3 se aplique hasta el inicio del siguiente período y no a la mitad de un pulso.

En CCER se dejó CC3P en 0 para que la salida sea activa en alto y se activó CC3E para sacar la señal por PB0. En CR1 se dejaron DIR y CMS en 0, con lo que el contador cuenta hacia arriba y la señal queda alineada al flanco. Por último, con el bit UG de EGR se cargaron PSC, ARR y CCR3 antes de arrancar, y con CEN se encendió el contador. Como el TIM3 es de propósito general, no tiene registro BDTR, así que no fue necesario activar MOE.

Configuración del ADC (Fig. 4)

Se habilitó el reloj del ADC con ADC12EN y en el registro común CCR se puso CKMODE en 01, para que el ADC use directamente el reloj del bus (HCLK/1), que es de 16 MHz.

Después se siguió la secuencia de encendido: se limpió DEEPPWD, se encendió ADVREGEN y se esperó con un ciclo for a que el regulador se estabilizara, ya que la documentación del ADC pide esperar al menos 20 µs. Esta vez también se hizo la calibración: con ADCALDIF en 0 se calibra en modo single-ended, que mide cada pin respecto a GND, y con ADCAL en 1 se inicia, esperando a que el hardware lo regrese a 0.

En SMPR1 se escribió 7 en los campos de los canales 1 y 2, que es el tiempo de muestreo máximo de 640.5 ciclos. Como el potenciómetro y el divisor del fotoresistor tienen una resistencia alta, un tiempo de muestreo largo le da más tiempo al capacitor interno del ADC para cargarse y la lectura sale más estable. Con 12 bits, la conversión agrega 12.5 ciclos, así que cada canal tarda:

$$t_{\text{conv}} = \frac{640.5 + 12.5}{16\ MHz} \approx 40.8\ \mu s$$

Es decir, la secuencia de los dos canales tarda unos 82 µs, mucho menos que lo que tarda una persona en girar la perilla o tapar el sensor.

En SQR1 se puso L = 1, que indica dos conversiones, SQ1 = 1 para que la primera sea PA0 y SQ2 = 2 para que la segunda sea PA1. En CFGR se dejó RES en 00 (12 bits), se activó CONT para que el ADC repita la secuencia sin parar y OVRMOD para que, si llega un dato nuevo sin haber leído el anterior, este se sobrescriba. En IER se activaron EOCIE y EOSIE, y con NVIC_EnableIRQ(ADC1_2_IRQn) se habilitó la interrupción en el NVIC. Al final se limpió ADRDY escribiendo un 1, se encendió ADEN, se esperó a que ADRDY se pusiera en 1 y se inició la primera conversión con ADSTART. Como el modo es continuo, solo hace falta poner ADSTART una vez.

Rutina de interrupción (Fig. 5)

Cada vez que el ADC termina una conversión o la secuencia completa se ejecuta ADC1_2_IRQHandler. Primero se lee ADC1_ISR una sola vez y se guarda en la variable estado, para revisar las dos banderas con el mismo valor:

- **EOC (bit 2):** se lee ADC1_DR, lo que también limpia la bandera, y el dato se guarda en lecturas[indice]. Luego indice aumenta en 1. La condición indice < 2 evita escribir fuera del arreglo si llegara un dato de más.
- **EOS (bit 3):** se limpia escribiendo un 1 en ISR y indice regresa a 0, para que la siguiente secuencia vuelva a empezar por el potenciómetro.

Al terminar el segundo canal, EOC y EOS se activan al mismo tiempo, por eso en una misma llamada se guarda el dato del fotoresistor y se reinicia el índice. Así, lecturas[0] siempre corresponde al canal 1 y lecturas[1] al canal 2.

Indicador de nivel (Fig. 6)

La función mostrar_nivel divide la lectura del potenciómetro en cinco niveles iguales de unas 819 cuentas cada uno:

$$LEDs = \frac{N \times 5}{4096}$$

Como es una división entera, el resultado es un número de 0 a 4 que indica cuántos LEDs se encienden. Por ejemplo, con la lectura máxima de 4095 queda 20475 / 4096 = 4.99, que se trunca a 4. Con esto quedan los siguientes rangos:

Lectura del ADC	Voltaje aprox.	LEDs encendidos
0 a 819	0 a 0.66 V	Ninguno
820 a 1638	0.66 a 1.32 V	PA12
1639 a 2457	1.32 a 1.98 V	PA12 y PA11
2458 a 3276	1.98 a 2.64 V	PA12, PA11 y PB5
3277 a 4095	2.64 a 3.3 V	Los cuatro

Los bits de los LEDs que deben encenderse se van juntando en encender_a y encender_b, y al final se escriben en el registro BSRR de cada puerto. La mitad alta del valor (bits 16 a 31) apaga los cuatro LEDs y la mitad baja (bits 0 a 15) enciende solo los que tocan. Si un mismo pin tiene en 1 su bit de apagado y su bit de encendido, el encendido tiene prioridad, así que con una sola escritura los LEDs quedan como deben, sin pasar por un estado apagado y sin modificar los demás pines del puerto.

Programa principal (Fig. 7)

El programa principal llama una vez a las funciones de configuración y luego entra al ciclo infinito. Ahí ya no se espera ninguna bandera del ADC, porque lecturas se actualiza sola en la interrupción. En el circuito, la lectura del fotoresistor baja cuando hay menos luz, así que si lecturas[1] es menor que UMBRAL_OSCURO se considera que está oscuro y se copia el valor del potenciómetro en CCR3. Como ARR es 4095, el ciclo de trabajo queda directamente proporcional al giro de la perilla:

$$D = \frac{lecturas[0]}{4096} \times 100\ \%$$

Si hay suficiente luz, CCR3 se pone en 0 y el LED de PB0 se mantiene apagado. En cualquiera de los dos casos se llama a mostrar_nivel, por lo que los cuatro LEDs siempre indican el nivel elegido con el potenciómetro, aunque el LED de PWM esté apagado.

Para el paso 4d se borraron las líneas del ciclo infinito. Según el código, en ese caso el ADC sigue convirtiendo y actualizando lecturas en la interrupción, pero nadie usa esos valores. El LED de PWM se queda con el valor inicial de CCR3 (50 % de brillo) y los cuatro LEDs de nivel quedan apagados, sin responder al potenciómetro ni al fotoresistor.

Circuito (Fig. 8)

En la Fig. 8 se muestra el circuito armado. Los cinco LEDs están conectados con sus resistencias limitadoras a PB0, PA12, PA11, PB5 y PB4. El potenciómetro está conectado entre 3.3 V y GND con el pin central en PA0, y el fotoresistor forma un divisor de voltaje con una resistencia fija, con el punto medio en PA1.

Conclusiones:

- Con esta práctica quedó claro que el brillo del LED no depende de cambiar el voltaje del pin, sino del tiempo que la señal pasa en alto en cada período. El pin solo entrega 0 V o 3.3 V, pero como la frecuencia es alta, el ojo promedia la señal y el LED se ve más o menos brillante según el ciclo de trabajo.

- Con esta práctica se comprendió mejor qué es una señal PWM y cómo funciona. En este caso se usó para regular el brillo de un LED, pero el mismo principio se aplica en muchas otras cosas, como controlar la velocidad de un motor DC, mover la posición de un servomotor, regular el voltaje en fuentes conmutadas o variar la intensidad de la iluminación LED. También quedó claro que, al generarla con un timer, la señal se mantiene sola por hardware y el CPU solo tiene que actualizar el registro CCR cuando quiere cambiar el ciclo de trabajo.

- Fue necesario depurar el programa para ver en tiempo real el valor que entregaba el ADC al leer el fotoresistor y así determinar un umbral que separara cuándo hay luz y cuándo no. Se revisó la lectura con la luz normal del ambiente y luego tapando el sensor con la mano, y el umbral se eligió entre esos dos valores. Esto dejó claro que el umbral depende de la iluminación del lugar donde se hace la prueba, por lo que un valor que funciona en un lugar podría no funcionar en otro ambiente y habría que volver a ajustarlo.

Bibliografía:

[1] STMicroelectronics, *STM32G4 Series Advanced Arm-Based 32-bit MCUs*, RM0440. Accedido el 28 de octubre de 2026. [En línea]. Disponible: *https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf*

[2] SparkFun Electronics, "Pulse Width Modulation," SparkFun Learn. Accedido el 28 de octubre de 2026. [En línea]. Disponible: *https://learn.sparkfun.com/tutorials/pulse-width-modulation/all*

[3] STMicroelectronics, *STM32 Cortex-M4 MCUs and MPUs Programming Manual*, PM0214. Accedido el 28 de octubre de 2026. [En línea]. Disponible: *https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf*

[4] Adafruit Industries, "Photocells," Adafruit Learning System. Accedido el 28 de octubre de 2026. [En línea]. Disponible: *https://learn.adafruit.com/photocells*

[5] L. Llamas, "The Potentiometer: Your Hand Enters the Circuit," Luis Llamas. Accedido el 28 de octubre de 2026. [En línea]. Disponible: *https://www.luisllamas.es/en/potentiometers-types-connection-logarithmic-linear/*